

WEEK 3 · CONVOLUTIONAL NEURAL NETWORKS

CNN 基礎：卷積數學、參數與反向傳播

Week 3 — 從一維卷積的定義出發，到 2D 卷積、CNN 的權重是什麼，再到卷積層如何做反向傳播

MLP 為什麼不夠？—— demo_mlp_limits.py 的結論

上一段用 `demo_mlp_limits.py` 和 `demo_mlp_limits_cifar.py` 已經示範了 MLP 的兩個根本問題：



沒有平移不變性

digit 位移幾個 pixel，準確率就崩掉 —— 每個像素對每個輸出都是「獨立」的權重，模型不知道位移後的圖其實是同一個東西。



權重長得像「模糊的位置樣板」

把第一層權重 reshape 回圖片形狀，看到的是位置固定、色塊模糊的 template，而不是可移動的邊緣偵測器。



參數量隨影像尺寸爆炸

Linear(3072, 512) 光第一層就有 1,572,864 個權重 —— 每個像素、每個輸出神經元都要一條獨立的線。

這一段要打開 CNN 的黑盒子：卷積這個運算到底在算什麼、它的權重是什麼、又是怎麼做反向傳播的。

從離散卷積到 2D 卷積

離散卷積 (1D) :

$$(f * g)[n] = \sum_m f[m] \cdot g[n - m]$$

把 g (kernel) 翻轉後滑過 f ，逐點相乘再加總

深度學習框架實際做的是「互相關」 (cross-correlation) :

$$Y[i,j] = \sum_p \sum_q X[i+p, j+q] \cdot W[p,q] + b$$

沒有翻轉 kernel — `nn.Conv2d` 的 'conv' 其實是這個

2D 卷積 = 一個小視窗，帶著權重，逐格滑過整張圖：

對照 `part1_conv_basics.py` 的 `sobel_kernels()`：本質上就是這個「逐格相乘再加總」的操作，只是 kernel 換成 Sobel。

輸入 (5×5)，紅框 = 目前的 3×3 視窗

7	2	0	2	3
7	4	0	3	5
7	6	8	1	7
0	5	2	1	5
2	5	2	1	6

卷積核 (weights) 3×3

1	0	-1
1	0	-1
1	0	-1

輸出特徵圖 (Feature Map) 3×3

13	6	-7
4	10	-7
-3	13	-6

輸出[1,1] = $\sum (\text{輸入} \times \text{權重}) = 4 \times 1 + 0 \times 0 + 3 \times -1 + 6 \times 1 + 8 \times 0 + 1 \times -1 + 5 \times 1 + 2 \times 0 + 1 \times -1 = 10$

Translation (shift) tolerance. Because it keeps only the *max* in each small window, small shifts in where a feature appears (e.g. an edge shifted by 1 pixel) often still produce the same max, so the output is less sensitive to *exactly* where a feature sits

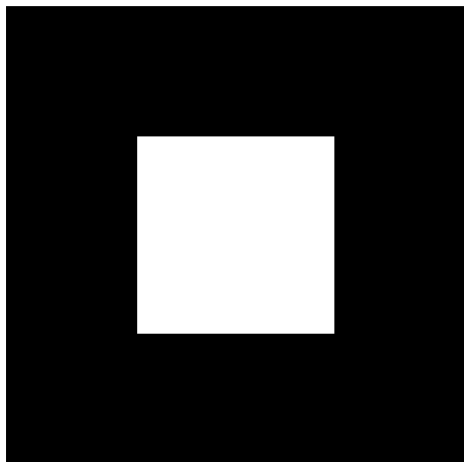
After a conv+ReLU, high values indicate strong feature presence ("this location strongly matches the kernel's pattern"). Max-pooling propagates the strongest signal forward and discards weaker ones nearby

The rule: during the forward pass, MaxPool remembers **which position** in each window held the max (like a switch/index). During backprop, the incoming gradient for that pooled output is sent entirely to that one winning position — every other position in the window gets a gradient of exactly **zero**, since changing them didn't affect the output at all.

手動設定 kernel : Sobel 邊緣偵測 (無需訓練)

卷積用一個小視窗滑過整張圖，偵測局部圖樣 (這裡手動設成 Sobel 邊緣偵測器)

輸入影像
(28×28 方形)



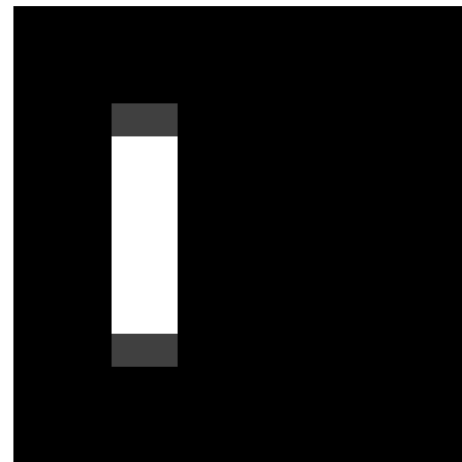
conv, kernel=Gx
(偵測垂直邊緣)



conv, kernel=Gy
(偵測水平邊緣)



ReLU + 2×2 池化
(14×14, 降採樣)



```
# part1_conv_basics.py
gx = torch.tensor([[-1,0,1],
                  [-2,0,2],
                  [-1,0,1]])
gy = gx.t().clone()

conv = nn.Conv2d(1, 2, 3, padding=1,
                 bias=False)
conv.weight[0,0] = gx # 垂直邊緣
conv.weight[1,0] = gy # 水平邊緣
edges = conv(img)
```

重點：這裡的權重是「人手設定」的，之後 part2/part3 CNN 裡的 kernel 是「訓練學出來」的——但運算方式完全一樣，只是數字換成可學習的參數。

輸出尺寸公式：卷積/池化後影像會縮小多少？

out = floor((in + 2×padding - kernel) / stride) + 1

input	kernel	stride	padding	output	意義
28	3	1	0	26	普通 3×3 conv, 尺寸縮小 2
28	3	1	1	28	3×3 + padding=1, 尺寸不變 (最常用)
28	5	1	0	24	kernel 變大, 縮得更多
28	2	2	0	14	MaxPool2d(2) : 尺寸減半
28	3	2	1	14	stride=2 的 conv, 效果等同 pooling

```
# part1_conv_basics.py
def conv_output_size(
    in_size, kernel,
    stride, padding):
    return (in_size
        + 2*padding - kernel)
        // stride + 1
```

這是設計 CNN 時必須自己追蹤的形狀 (Conv2d(8,7,3,padding=1) → MaxPool2d(2) → Flatten 之前一定要算對)。

For any sliding-window operation (convolution **or** pooling) along one dimension:

$$\text{Output size} = \left\lfloor \frac{\text{Input size} - \text{Kernel size} + 2 \times \text{Padding}}{\text{Stride}} \right\rfloor + 1$$

Written with symbols — W = input size, K = kernel size, P = padding, S = stride:

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1$$

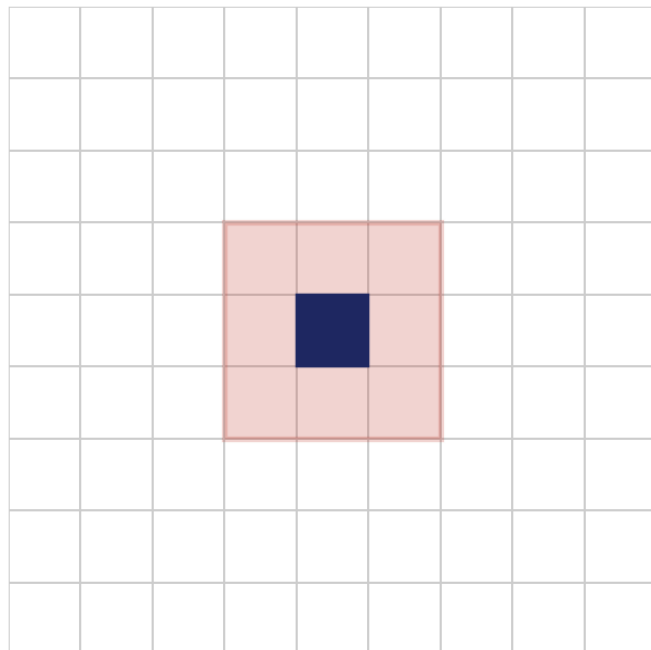
Your MaxPool(2): kernel size 2, stride 2 (stride defaults to equal the kernel size for pooling), padding 0, $W = 28$:

$$O = \left\lfloor \frac{28 - 2 + 0}{2} \right\rfloor + 1 = \lfloor 13 \rfloor + 1 = 14$$

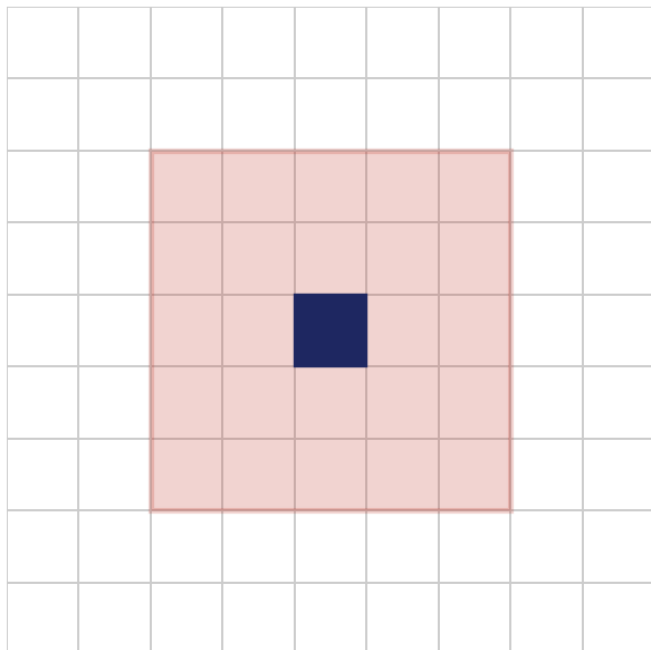
疊得越深，看得越廣：Receptive Field

感受野 (Receptive Field)：層疊越深，每個輸出像素「看到」的輸入範圍越大

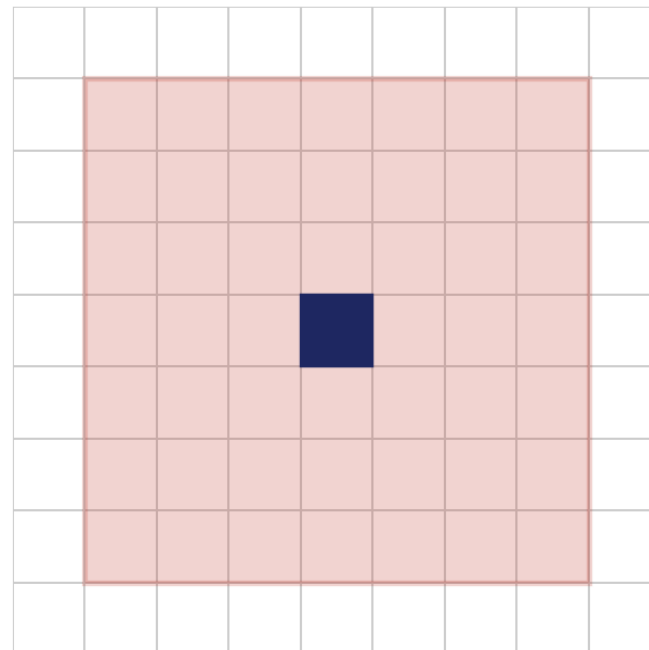
疊 1 層 3×3 conv
輸出的 1 個像素看到輸入的 3×3



疊 2 層 3×3 conv
輸出的 1 個像素看到輸入的 5×5



疊 3 層 3×3 conv
輸出的 1 個像素看到輸入的 7×7



$rf1 = 3$

$rf2 = rf1 + (3 - 1) = 5$

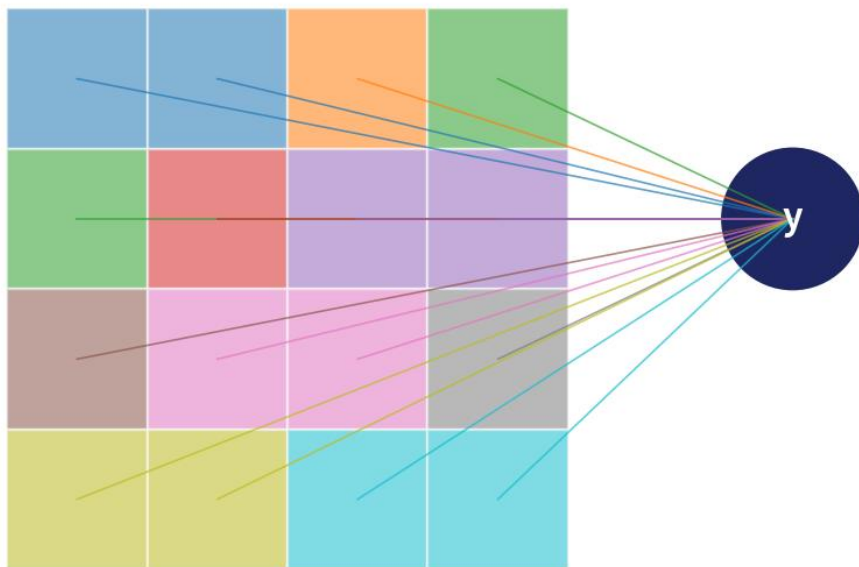
一層 3×3 conv 看到 3×3

疊第二層 3×3 conv 看到 5×5

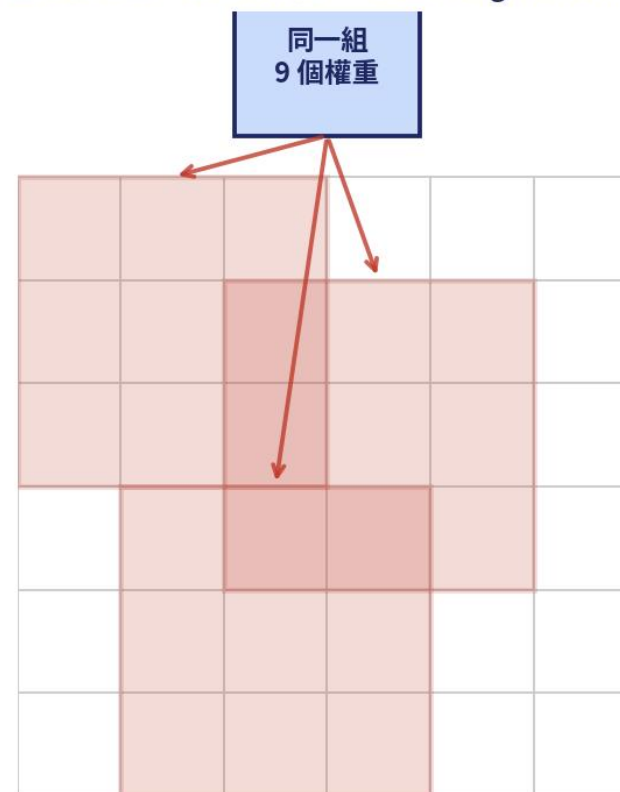
Weight Sharing：同一組核，重複用在每個位置

CNN 的權重是什麼？—— 一組被「共享」的小濾波器，而不是每個像素一個獨立權重

Dense (全連接)
每個輸入像素 → 這個輸出 都是「獨立」的權重
(16 個像素 = 16 個不同的 w)



Conv (卷積)
同一個 3×3 核 (9 個權重)
在每個位置「重複使用」—— 這叫 weight sharing



一個 Conv2D 層裡到底存了幾個數字？

參數量 = out_ch × (in_ch × kH × kW + 1) = 32 × (3 × 3 × 3 + 1) = 896



```
nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3)
```

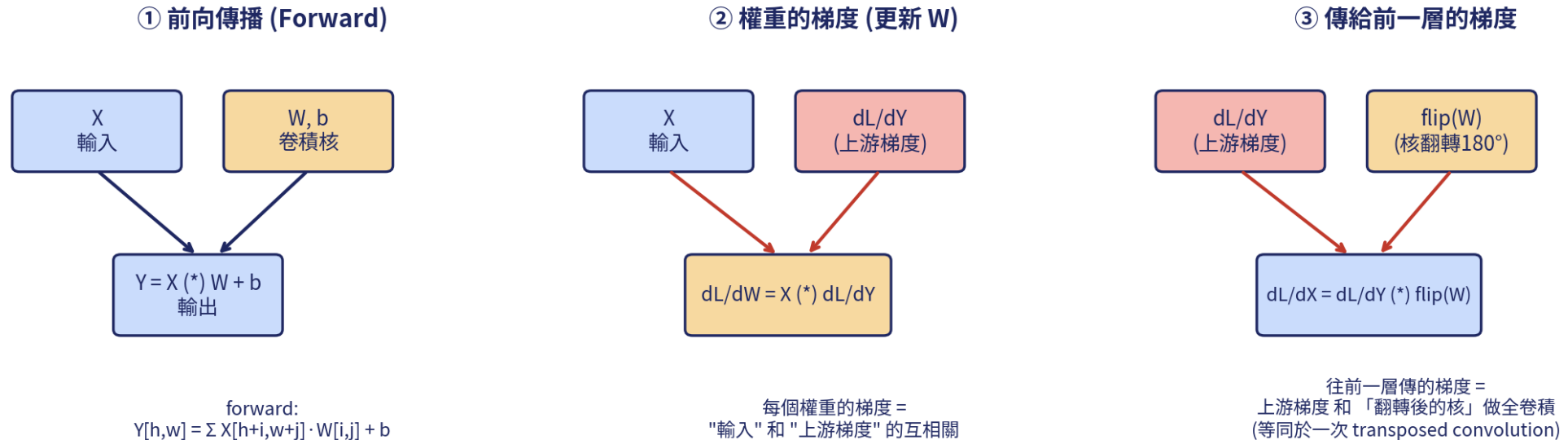
每個 filter 形狀 = (in_channels=3, kH=3, kW=3)，共 32 個 filter + 32 個 bias

層	程式	參數量
Dense (Week2 CIFAR MLP)	nn.Linear(3072, 512)	1,572,864
Conv (part3_cnn_cifar10)	nn.Conv2d(3, 32, 3)	896
Conv (part3_cnn_cifar10)	nn.Conv2d(32, 64, 3)	18,496

同一組 9 個(xchannel) 權重被整張圖共用，參數量只跟filter 大小/數量有關，跟輸入影像尺寸「無關」。

Conv2D 怎麼做 backprop ? —— 兩個梯度都是「再卷積一次」

Conv2D 的反向傳播：兩個梯度都可以寫成「再做一次卷積」



```
loss.backward() # part2/3 只需要這一行 — autograd 自動算出上面兩個梯度
# 手刻版本 (呼應 week1 part1_manual_gradient.py 的精神) :
dW = einsum('nohw,nchwij->ocij', dOut, input_windows) # 對應 X (*) dL/dY
dX = 對每個 kernel 位移 (i,j), 把 dOut @ W[:, :, i, j] 加回 dX 對應的位置
```

這一段建立的四個關鍵概念

1

卷積 = 滑動視窗做加權和

$Y[i,j] = \sum X[i+p,j+q] \cdot W[p,q] + b$ ，與 Dense 的差別在於「同一組權重」被重複使用。

2

形狀公式要背

$out = \text{floor}((in+2p-k)/s)+1$ ，設計網路時每一層都要對得上。

3

參數量不隨影像尺寸增加

3×3 conv 不管圖多大，永遠只有 $9 \times in_ch \times out_ch + out_ch$ 個數字。

4

backprop 本身也是卷積

dW 是輸入和上游梯度的互相關； dX 是上游梯度和「翻轉後的核」做全卷積。

下一段：[part2_cnn_mnist.py](#) —— 把這些 Conv 疊成完整的 CNN，在 MNIST 上實際跑起來，並和 MLP 的參數量/準確率對比。